Thông tin chung

SDN302 _PE_SUMMER26
Subject: SDN302
Duration: 85 minutes

INSTRUCTIONS

Read the instructions below carefully before starting the exam.

- You are **NOT allowed** to use any device to share data with others.
- For your development tools, you must use **Visual Studio Code, MongoDBCompass, Postman, MongoDB V6 or later**.
- **studentcode:** e.g., se181234
- **Yourdomain:** hostname+port; e.g., localhost:5000

IMPORTANT – before you start doing your solution, you MUST carry out the following steps:

1. Before submitting your work, check that the submitted PE contains no syntax error(s).
2. The data should be obtained from the provided JSON files.
3. **If there are syntax errors or compilation errors in your PE program, you will not pass the PE requirements, the mark will be 0.**
4. Create a new project in Visual Studio Code named **sdn302-pe-su26-studentcode**
5. You must create Mongo database with the name **sdn302-pe-su26-studentcode-db**
6. **Your submission will be invalid (0 points) if it contains irrelevant content. Store your MongoDB connection string, port value, and JWT secret key in the .env file. Failure to do so will result in 0 points.**

Scenario

You will implement a part of the Helicopter Management System. The scenario is as follows: a manufacturer owns many Helicopters, and each Helicopter belongs to one manufacturer.

Database schema

Your database contains 3 collections with the following schema:

```
manufacturers {

    manufacturer: {type: String, required: true, unique: true},

    isAsia: {type: Boolean, default: false},

}, {timestamps: true});

helicopters{

    heliName: {type: String, required: true, unique: true},

    imgWrapper: {type: String, required: true},

    isAssembly: {type: Boolean, default: false},

    price: {type: Number, required: true},

    adult: {type: Boolean, default: false},

    manufacturer: {type: mongoose.Schema.Types.ObjectId, ref: "manufacturer", required: true},

}, { timestamps: true, });

members {

    name: {type: String, required: true},

    pass: {type: String, required: true},

}, {timestamps: true});
```

Task 01 (1.0 mark)

Using the Express Generator, create a REST API server named **sdn302-pe-su26-studentcode**, then create the related manufacturers and migrate them to the MongoDB database **sdn302-pe-su26-studentcode-db**.

Task 02 (5.0 marks)

Build a new REST API server for the manufacturer collection. You must use JSON Web Token (JWT) for authenticating the API. The JWT secret key must be your StudentCode (e.g., se181234@) and stored in the `.env` file. The route path will match requests to:

- Visit http://yourdomain/auth/tokens to generate the JWT. The account credentials are stored in the members.json file (name: soldier, pass: 123456789). Note: Do not store the JWT in cookies or session storage.
- http://yourdomain/api/manufacturers and http://yourdomain/api/manufacturers/:id for all methods related to manufacturers.
- Implement JWT-protected CRUD operations for the manufacturer collection.
- *For the DELETE operation, ensure a manufacturer cannot be deleted if any helicopters are associated with it (i.e., if there are documents in the Helicopter collection referencing the manufacturer's _id). Return an appropriate error message (e.g., "Cannot delete manufacturer because it has associated Helicopters") if deletion is attempted in such cases.*

— Tài liệu được thành viên đăng tải tại **FUOVERFLOW** và được biên soạn lại để hỗ trợ ôn tập —

Task 03 (4.0 marks)

Client Application for Helicopter Management

You must choose one of the template engines (EJS, HANDLEBARS, or PUG) with Express for building the views.

1. Authentication

On the login view, the route path will match requests to: http://yourdomain/auth/access. Users must enter their account name (name) and pass (pass) correctly.

- The pass must be encrypted using bcrypt and stored in the members collection.
- All CRUD actions require authentication.
- Show an error message for failed login attempts. Redirect to the Helicopter list after a successful login.
- Import the account data from members.json into the members collection.

2. Display Helicopter List

After successful login:

- Display all helicopters in a card view (not a table view), including: heliName, price (displayed as the price in USD, calculated as price/26000 and rounded to one decimal place. E.g., { "price": 1709887 } → 65.8 USD), imgWrapper, adult (displayed as a male (true)/child (false) icon), and manufacturer name.

The route path: http://yourdomain/management/helicopters

3. Delete Helicopter

After successful login:

- Allow the user to delete a selected helicopter with confirmation.
- Display a notification message showing the result.

The route path: http://yourdomain/management/helicopters/:id

4. Add New Helicopter

After successful login, a new Helicopter can be added using a modal form that meets the requirements. All fields are required.

- *heliName* must contain at least two words, and the first letter of each word must be capitalized.
- *imgWrapper* must be url.
- *price* must be a number formatted with a minimum of 6 digits.
- *adult* must be a toggle or switch control, *adult* can be true or false.
- *isAssembly* must be a toggle or switch control. If *adult* is true, then *isAssembly* must always be true.
- *manufacturer* must be a select control displaying manufacturerName values from the manufacturers collection.

The route path: http://yourdomain/management/helicopters

5. Update Helicopter

After successful login:

- All controls must meet the same requirements as the add function.

The route path for viewing and updating the detail of a helicopter is:
http://yourdomain/management/helicopters/:id

-The end of test-